

Fake News Detection Project

A step-by-step guide from raw data to a working demo

1. Define Scope & Gather Data

Decide whether you're detecting fake news from article text only, or including images, social context, or metadata. Pick a dataset (e.g., the Kaggle True.csv/Fake.csv set for a simple start, or FakeNewsNet, LIAR, or FineFake for more advanced work). Download and inspect the data for class balance, missing values, and duplicate entries.

2. Clean & Preprocess Text

Lowercase text, remove HTML tags, URLs, punctuation, and stopwords. Apply tokenization and lemmatization/stemming. Merge title and body text if useful. Split into train/validation/test sets (a common split is 70/15/15), keeping class balance consistent across splits.

3. Explore the Data (EDA)

Look at word frequency and length distributions for fake vs. real articles, check for source/domain leakage (e.g., all real news from one outlet), and visualize class distribution. This step often reveals biases that could make your model unrealistically accurate.

4. Feature Engineering

Convert text into numerical features: start with TF-IDF or bag-of-words for a baseline model. For deeper models, use word embeddings (Word2Vec, GloVe) or contextual embeddings from a pretrained transformer (BERT, RoBERTa). If using multimodal data, extract image features separately (e.g., with a CNN).

5. Build Baseline Models

Train simple, interpretable models first: Logistic Regression, Naive Bayes, or SVM on TF-IDF features. This gives you a quick benchmark and confirms your pipeline works end-to-end before adding complexity.

6. Build Advanced Models

Move to deep learning if the baseline isn't accurate enough: LSTM/BiLSTM or CNN on embeddings, or fine-tune a pretrained transformer like BERT/RoBERTa for text classification. For multimodal datasets, combine text and image branches into a fusion model.

7. Evaluate Rigorously

Use accuracy, precision, recall, F1-score, and a confusion matrix — not just accuracy, since fake/real classes can be imbalanced. Check performance across different topics or sources (cross-domain

evaluation) to see if the model generalizes or just memorizes surface patterns.

8. Deploy & Present Results

Wrap the trained model in a simple interface (Streamlit, Flask, or a notebook demo) where a user pastes an article and gets a prediction with a confidence score. Document your pipeline, dataset limitations, and known biases in a README or report — this matters a lot for credibility in this domain.

Practical Tips

- **Start simple.** A TF-IDF + Logistic Regression baseline can hit 90%+ accuracy on the Kaggle True/Fake dataset — get that working before reaching for transformers.
- **Watch for data leakage.** If all “real” articles come from Reuters and all “fake” ones come from other sites, your model may just be learning to detect writing style or source, not actual truthfulness. This is a well-known issue with several public datasets.
- **Suggested stack:** Python with pandas and scikit-learn, nltk or spaCy for preprocessing, and Hugging Face's transformers library if you go the BERT route.